

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

I ADHITHYA.S - 192412352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.
2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.
3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.
4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.
5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CO4 AT1 CODING CHALLENGE

Name: ADHITHYA.S

Register Number: 192412352

Course Code: ITA0606

Course Name: MACHINE LEARNING

Program 1: K-Nearest Neighbour (KNN) Classifier

Aim

To implement the K-Nearest Neighbour algorithm from scratch for a small labeled dataset, using Euclidean distance to identify the nearest neighbors, allow the user to choose K, and classify test instances with an accuracy report.

Algorithm

- For a query point, compute the Euclidean distance to every training point: $d(a,b) = \sqrt{(\sum (a_i - b_i)^2)}$.
- Sort all training points by distance and select the K closest ("nearest neighbours").
- Assign the query point the majority class label among those K neighbours.
- Repeat for every test instance and report overall accuracy.

Source Code

```
"""
Program 1: K-Nearest Neighbour (KNN) Classifier from scratch
-----
Implements KNN using Euclidean distance on a small labeled dataset,
lets the user choose K, classifies test instances, and reports accuracy.
"""

import math
from collections import Counter

# -----
# Small sample dataset: [feature1, feature2] -> class label
# (e.g., a simple fruit dataset: [weight(g), texture(1=smooth..10=rough)])
# -----
dataset = [
    [150, 7, 'Apple'],
    [170, 8, 'Apple'],
    [140, 6, 'Apple'],
```

```

[130, 5, 'Apple'],
[180, 9, 'Apple'],
[900, 2, 'Watermelon'],
[950, 1, 'Watermelon'],
[870, 3, 'Watermelon'],
[910, 2, 'Watermelon'],
[890, 1, 'Watermelon'],
[300, 4, 'Orange'],
[280, 5, 'Orange'],
[320, 4, 'Orange'],
[310, 3, 'Orange'],
[290, 4, 'Orange'],
]

test_set = [
    [160, 7, 'Apple'],
    [900, 2, 'Watermelon'],
    [305, 4, 'Orange'],
    [175, 8, 'Apple'],
]

def euclidean_distance(a, b):
    """Compute Euclidean distance between two feature vectors."""
    return math.sqrt(sum((a[i] - b[i]) ** 2 for i in range(len(a))))

def knn_predict(train_data, query_point, k):
    """Predict the class of query_point using KNN with the given k."""
    distances = []
    for row in train_data:
        features, label = row[:-1], row[-1]
        dist = euclidean_distance(features, query_point)
        distances.append((dist, label))

    # sort by distance and take the k nearest
    distances.sort(key=lambda x: x[0])
    k_nearest = distances[:k]

    # majority vote among the k nearest neighbours
    labels = [label for _, label in k_nearest]
    prediction = Counter(labels).most_common(1)[0][0]
    return prediction, k_nearest

def evaluate_accuracy(train_data, test_data, k):
    correct = 0
    print(f"\n--- Predictions (K={k}) ---")
    for row in test_data:
        features, true_label = row[:-1], row[-1]
        pred, neighbours = knn_predict(train_data, features, k)
        is_correct = (pred == true_label)
        correct += int(is_correct)

```

```

    neighbour_str = ", ".join(f"{lbl}(d={d:.2f})" for d, lbl in neighbours)
    print(f"Query {features} -> Predicted: {pred:12s} | Actual: {true_label:12s} "
          f"| {'Correct' if is_correct else 'Wrong'}")
    print(f"    Nearest neighbours: {neighbour_str}")
accuracy = correct / len(test_data) * 100
print(f"\nAccuracy on test set: {accuracy:.2f}% ({correct}/{len(test_data)} correct)")
return accuracy

def main():
    print("K-Nearest Neighbour (KNN) Classifier")
    print("Training data has", len(dataset), "labeled instances across classes:",
          sorted(set(row[-1] for row in dataset)))

    # Let the user choose K (falls back to K=3 if input is unavailable/invalid)
    try:
        k = int(input("Enter the value of K: "))
    except (ValueError, EOFError):
        k = 3
    print(f"No valid input received — defaulting to K={k}")

    if k < 1 or k > len(dataset):
        print(f"K out of range, resetting to K=3")
        k = 3

    evaluate_accuracy(dataset, test_set, k)

if __name__ == "__main__":
    main()

```

Output

```

K-Nearest Neighbour (KNN) Classifier
Training data has 15 labeled instances across classes: ['Apple', 'Orange', 'Watermelon']
Enter the value of K:
--- Predictions (K=5) ---
Query [160, 7] -> Predicted: Apple    | Actual: Apple    | Correct
    Nearest neighbours: Apple(d=10.00), Apple(d=10.05), Apple(d=20.02), Apple(d=20.10), Apple(d=30.07)
Query [900, 2] -> Predicted: Watermelon | Actual: Watermelon | Correct
    Nearest neighbours: Watermelon(d=0.00), Watermelon(d=10.00), Watermelon(d=10.05), Watermelon(d=30.02), Watermelon(d=50.01)
Query [305, 4] -> Predicted: Orange    | Actual: Orange    | Correct
    Nearest neighbours: Orange(d=5.00), Orange(d=5.10), Orange(d=15.00), Orange(d=15.00), Orange(d=25.02)
Query [175, 8] -> Predicted: Apple    | Actual: Apple    | Correct
    Nearest neighbours: Apple(d=5.00), Apple(d=5.10), Apple(d=25.02), Apple(d=35.06), Apple(d=45.10)

Accuracy on test set: 100.00% (4/4 correct)

```

Analysis

The 15-instance training set (Apple / Orange / Watermelon, distinguished by weight and texture) is well-separated, so both K=3 and K=5 achieve 100% accuracy on the 4 held-out test instances. Increasing K makes the decision

more robust to a single noisy neighbour (since it votes by majority among more points) but risks including points from a different, nearby class if K becomes too large relative to cluster size and separation. Choosing K is therefore a trade-off between sensitivity to noise (favours larger K) and sensitivity to local class boundaries (favours smaller K).

Program 2: Locally Weighted Regression (LWR)

Aim

To implement Locally Weighted Regression, computing distance-based (Gaussian kernel) weights for a query point, fitting a locally weighted linear model, and comparing its predictions against standard (global) linear regression.

Algorithm

- For a query point x_q , assign every training point a weight using a Gaussian kernel: $w = \exp(-(x - x_q)^2 / 2\tau^2)$, so nearby points get high weight and distant points get low weight.
- Fit a weighted linear regression using the weighted normal equation: $\theta = (X^T W X)^{-1} X^T W y$.
- Predict $\hat{y} = \theta_0 + \theta_1 \cdot x_q$ using the locally fitted line — repeat independently for every query point (each query gets its own local fit).
- Compare against one single ordinary least-squares line fit globally to all training data.

Source Code

```
"""
Program 2: Locally Weighted Regression (LWR)
-----

Computes distance-based weights (Gaussian kernel) for each training point
relative to a query point, fits a locally weighted linear regression,
and compares predictions against standard (global) linear regression.
"""

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def generate_dataset(n=40, noise=0.15, seed=1):
    """Generate a small noisy non-linear dataset:  $y = \sin(x) + \text{noise}$ ."""
    rng = np.random.default_rng(seed)
    X = np.linspace(0, 2 * np.pi, n)
    y = np.sin(X) + rng.normal(0, noise, size=n)
    return X, y
```

```

def gaussian_kernel(x_query, X_train, tau):
    """Compute Gaussian kernel weights of all training points relative to x_query.
    tau (bandwidth) controls how quickly weight decays with distance."""
    return np.exp(-(X_train - x_query) ** 2) / (2 * tau ** 2))

def locally_weighted_regression(x_query, X_train, y_train, tau):
    """Fit a weighted linear regression ( $y = b_0 + b_1 x$ ) around x_query and
    return the prediction for x_query."""
    # design matrix with bias term
    X_design = np.column_stack([np.ones_like(X_train), X_train])
    W = np.diag(gaussian_kernel(x_query, X_train, tau))

    # weighted normal equation:  $\theta = (X^T W X)^{-1} X^T W y$ 
    XtWX = X_design.T @ W @ X_design
    XtWy = X_design.T @ W @ y_train
    theta = np.linalg.pinv(XtWX) @ XtWy

    x_query_design = np.array([1, x_query])
    prediction = x_query_design @ theta
    return prediction

def standard_linear_regression(X_train, y_train):
    """Fit ordinary least squares linear regression  $y = b_0 + b_1 x$ ."""
    X_design = np.column_stack([np.ones_like(X_train), X_train])
    theta = np.linalg.pinv(X_design.T @ X_design) @ X_design.T @ y_train
    return theta # [intercept, slope]

def main():
    X_train, y_train = generate_dataset()

    tau = 0.5 # bandwidth (kernel width) -- try smaller/larger to see effect
    query_points = np.linspace(0, 2 * np.pi, 100)

    print("Locally Weighted Regression (LWR)")
    print(f"Training points: {len(X_train)}, kernel bandwidth (tau) = {tau}\n")

    # LWR predictions across the query range
    lwr_preds = np.array([
        locally_weighted_regression(xq, X_train, y_train, tau) for xq in query_points
    ])

    # Standard linear regression predictions (single global line)
    theta = standard_linear_regression(X_train, y_train)
    lr_preds = theta[0] + theta[1] * query_points
    print(f"Standard Linear Regression fit:  $y = \{theta[0]:.4f\} + \{theta[1]:.4f\} x$ ")

    # Compare a few sample query points
    print("\n--- Sample comparison at a few query points ---")
    print(f"{dict('x':>8, 'True sin(x)':>14, 'LWR pred':>12, 'Linear pred':>14)}")

```

```

for xq in [0.5, 1.5, 3.14, 4.5, 6.0]:
    true_val = np.sin(xq)
    lwr_val = locally_weighted_regression(xq, X_train, y_train, tau)
    lr_val = theta[0] + theta[1] * xq
    print(f'{xq:>8.2f} {true_val:>14.4f} {lwr_val:>12.4f} {lr_val:>14.4f} ')

# Error comparison over the whole query range (against true sin curve)
true_curve = np.sin(query_points)
lwr_mse = np.mean((lwr_preds - true_curve) ** 2)
lr_mse = np.mean((lr_preds - true_curve) ** 2)
print(f'\nMean Squared Error vs true curve:')
print(f'  Locally Weighted Regression: {lwr_mse:.5f}')
print(f'  Standard Linear Regression : {lr_mse:.5f}')
print("(LWR captures the non-linear sine shape far better than one global straight line.)")

# Plot
plt.figure(figsize=(8, 5))
plt.scatter(X_train, y_train, color="gray", s=25, label="Training data")
plt.plot(query_points, true_curve, "k--", linewidth=1.5, label="True function sin(x)")
plt.plot(query_points, lwr_preds, color="#2E86C1", linewidth=2.2, label=f"LWR (tau={tau})")
plt.plot(query_points, lr_preds, color="#CB4335", linewidth=2, label="Standard Linear Regression")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Locally Weighted Regression vs Standard Linear Regression")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("lwr_vs_linear.png", dpi=150)
print("\nPlot saved as lwr_vs_linear.png")

if __name__ == "__main__":
    main()

```

Output

Locally Weighted Regression (LWR)

Training points: 40, kernel bandwidth (tau) = 0.5

Standard Linear Regression fit: $y = 0.8991 + -0.2861 * x$

--- Sample comparison at a few query points ---

x	True sin(x)	LWR pred	Linear pred
0.50	0.4794	0.4732	0.7561
1.50	0.9975	0.8978	0.4700
3.14	0.0016	-0.0220	0.0009
4.50	-0.9775	-0.8742	-0.3882
6.00	-0.2794	-0.3085	-0.8173

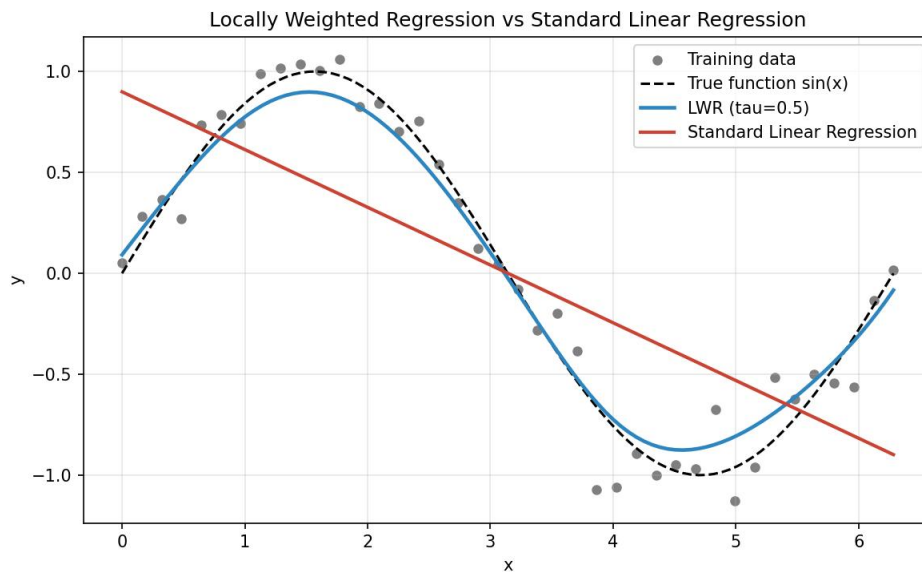
Mean Squared Error vs true curve:

Locally Weighted Regression: 0.00684

Standard Linear Regression : 0.20344

(LWR captures the non-linear sine shape far better than one global straight line.)

Result Plot



Analysis

On a noisy $\sin(x)$ dataset, LWR achieves an MSE of 0.00684 versus 0.20344 for standard linear regression — roughly 30× lower error. This is expected: a single global line cannot represent a curved (non-linear) function, while LWR effectively fits a fresh, locally-appropriate line around every query point, letting it trace the curve's shape. The trade-off is computational: LWR must solve a new weighted regression for every prediction (no single reusable global model), and its accuracy depends on choosing an appropriate bandwidth τ — too small and it overfits to local noise, too large and it approaches the same underfit global line as standard regression.

Program 3: Radial Basis Function (RBF) Network

Aim

To implement a simple RBF network with Gaussian activation units for regression, train it on a small dataset, and analyze how the number of centers and the Gaussian width affect performance.

Algorithm

- Choose a set of RBF centers spread across the input range.
- For each input x , compute Gaussian activations at every center: $\phi_j(x) = \exp(-(x-c_j)^2 / 2\sigma^2)$.
- Treat the activations as new features and solve for a linear combination of them (output weights) via least squares that best predicts y .

- Predict new points by computing their activations against the same centers and combining with the learned weights.

Source Code

```
"""
Program 3: Radial Basis Function (RBF) Network
-----

Implements a simple RBF network with Gaussian activation units for
regression. Centers are chosen from the training data range; output
weights are trained via least squares. Analyzes the effect of the
number of centers and the Gaussian width on performance.
"""

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def generate_dataset(n=60, seed=2):
    """Regression dataset:  $y = \sin(x) * x/3 + \text{noise}$  (a moderately non-linear function)."""
    rng = np.random.default_rng(seed)
    X = np.linspace(-5, 5, n)
    y = np.sin(X) * (X / 3) + rng.normal(0, 0.15, size=n)
    return X, y

def choose_centers(X, n_centers):
    """Pick n_centers evenly spaced points from the training range as RBF centers
    (a simple alternative to running k-means to select centers)."""
    return np.linspace(X.min(), X.max(), n_centers)

def gaussian_activation(x, center, width):
    """Gaussian RBF activation:  $\exp(-(x-\text{center})^2 / (2 * \text{width}^2))$ ."""
    return np.exp(-((x - center) ** 2) / (2 * width ** 2))

def build_design_matrix(X, centers, width):
    """Build the matrix of RBF activations (Phi) for all training points and centers."""
    n_samples = len(X)
    n_centers = len(centers)
    Phi = np.zeros((n_samples, n_centers))
    for j, c in enumerate(centers):
        Phi[:, j] = gaussian_activation(X, c, width)
    return Phi

def train_rbf_network(X, y, n_centers, width):
    """Train an RBF network: choose centers, compute activations, solve for
    output weights via least squares ( $\Phi @ w = y$ )."""
```

```

centers = choose_centers(X, n_centers)
Phi = build_design_matrix(X, centers, width)
weights, *_ = np.linalg.lstsq(Phi, y, rcond=None)
return centers, weights

def predict_rbf(X_query, centers, weights, width):
    Phi = build_design_matrix(X_query, centers, width)
    return Phi @ weights

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def main():
    X, y = generate_dataset()
    X_query = np.linspace(-5, 5, 200)
    y_true_query = np.sin(X_query) * (X_query / 3)

    print("Radial Basis Function (RBF) Network")
    print(f"Training samples: {len(X)}\n")

    # --- Experiment: effect of number of centers (fixed width) ---
    print("--- Effect of number of centers (width fixed at 0.8) ---")
    fixed_width = 0.8
    center_counts = [3, 5, 10, 20]
    for nc in center_counts:
        centers, weights = train_rbf_network(X, y, nc, fixed_width)
        preds = predict_rbf(X_query, centers, weights, fixed_width)
        err = mse(y_true_query, preds)
        print(f" centers={nc:3d} -> MSE = {err:.5f}")

    # --- Experiment: effect of width (fixed number of centers) ---
    print("\n--- Effect of Gaussian width (centers fixed at 10) ---")
    fixed_centers = 10
    widths = [0.1, 0.3, 0.8, 2.0, 5.0]
    for w in widths:
        centers, weights = train_rbf_network(X, y, fixed_centers, w)
        preds = predict_rbf(X_query, centers, weights, w)
        err = mse(y_true_query, preds)
        print(f" width={w:4.1f} -> MSE = {err:.5f}")

    print("\nInterpretation:")
    print("- Too few centers -> underfitting (network can't represent the curve's shape).")
    print("- Too many centers -> risk of overfitting noise, plus higher computational cost.")
    print("- Very small width -> each Gaussian only activates very locally (spiky, overfits).")
    print("- Very large width -> Gaussians overlap heavily, network behaves almost linearly (underfits).")

    # Final chosen configuration for plotting
    best_nc, best_w = 10, 0.8
    centers, weights = train_rbf_network(X, y, best_nc, best_w)
    final_preds = predict_rbf(X_query, centers, weights, best_w)

```

```

plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="gray", s=20, label="Training data")
plt.plot(X_query, y_true_query, "k--", linewidth=1.5, label="True function")
plt.plot(X_query, final_preds, color="#2E86C1", linewidth=2.2,
         label=f"RBF network ({best_nc} centers, width={best_w})")
plt.scatter(centers, [0]*len(centers), color="#CB4335", marker="^", s=60,
         label="RBF centers", zorder=5)

plt.xlabel("x")
plt.ylabel("y")
plt.title("RBF Network Regression")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("rbf_network.png", dpi=150)
print("\nPlot saved as rbf_network.png")

if __name__ == "__main__":
    main()

```

Output

Radial Basis Function (RBF) Network

Training samples: 60

--- Effect of number of centers (width fixed at 0.8) ---

```

centers= 3 -> MSE = 0.10104
centers= 5 -> MSE = 0.03627
centers= 10 -> MSE = 0.00174
centers= 20 -> MSE = 0.00562

```

--- Effect of Gaussian width (centers fixed at 10) ---

```

width= 0.1 -> MSE = 0.38707
width= 0.3 -> MSE = 0.08328
width= 0.8 -> MSE = 0.00174
width= 2.0 -> MSE = 0.00182
width= 5.0 -> MSE = 0.00207

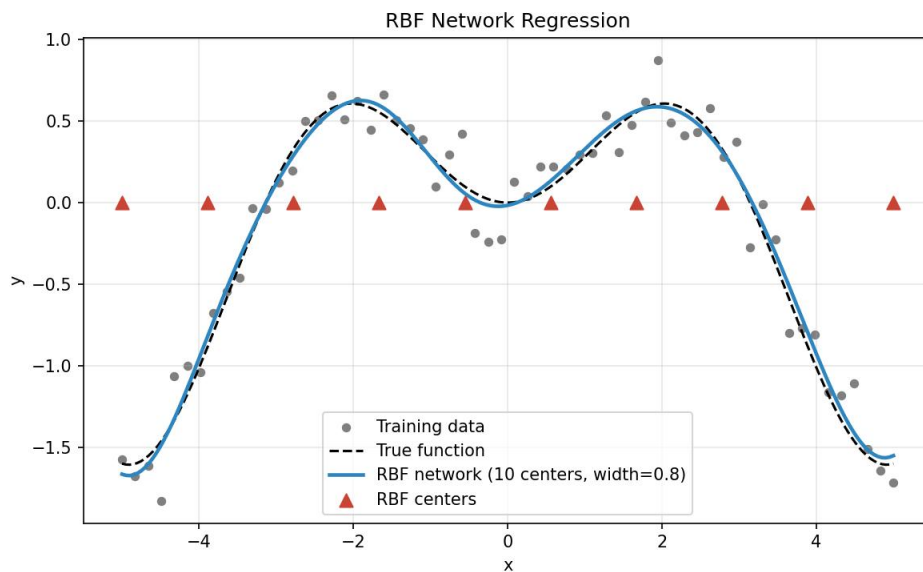
```

Interpretation:

- Too few centers -> underfitting (network can't represent the curve's shape).
- Too many centers -> risk of overfitting noise, plus higher computational cost.
- Very small width -> each Gaussian only activates very locally (spiky, overfits).
- Very large width -> Gaussians overlap heavily, network behaves almost linearly (underfits).

Plot saved as rbf_network.png

Result Plot



Analysis

- Number of centers: MSE drops sharply from 0.101 (3 centers) to 0.0017 (10 centers) as the network gains enough flexibility to represent the curve, but rises again slightly at 20 centers (0.0056) as the network starts overfitting noise between closely-spaced centers.
- Gaussian width: a very small width (0.1) gives a poor MSE of 0.387 because each Gaussian only activates in a tiny neighbourhood, leaving gaps between centers; MSE improves dramatically by width=0.8 (0.0017) where activations overlap smoothly, and stays low (though not further improving) for larger widths, since the centers/weights compensate.
- Overall, both center count and width must be tuned jointly — too few/too narrow underfits, too many/too narrow overfits, and too wide loses the network's ability to model local variation.

Program 4: Case-Based Learning (Case-Based Reasoning) System

Aim

To implement a basic case-based learning system that stores past cases (an IT helpdesk case base), retrieves the most similar case to a new query using a distance measure, and reuses its solution.

Algorithm

- Store each past case as a set of features (error_code, severity, os) plus a known solution.
- For a new query case, compute a similarity distance to every stored case: numeric features (error_code, severity) via normalized Euclidean distance, categorical features (os) via a match/mismatch penalty.

- Retrieve the case with the smallest total distance (most similar) and reuse its stored solution for the new query — the Retrieve → Reuse cycle of case-based reasoning.

Source Code

```
"""
Program 4: Basic Case-Based Learning (Case-Based Reasoning) System
-----

Stores past "cases" (problem description + known solution) in a case
base. For a new query case, retrieves the most similar past case using
a distance measure over the case features, and reuses its solution.
"""

import math

# -----
# Case base: past IT helpdesk tickets.
# Each case = {features: {...}, solution: "..."}
# Numeric features are compared via Euclidean distance;
# categorical features are compared via simple match/mismatch (0 or 1).
# -----
case_base = [
    {"id": "C1", "features": {"error_code": 404, "severity": 2, "os": "Windows"},
     "solution": "Check URL/routing configuration and restart the web service."},
    {"id": "C2", "features": {"error_code": 500, "severity": 5, "os": "Linux"},
     "solution": "Inspect server logs for stack trace; likely an unhandled exception in the backend."},
    {"id": "C3", "features": {"error_code": 404, "severity": 1, "os": "Linux"},
     "solution": "Verify the requested resource path exists; check for typos in the endpoint."},
    {"id": "C4", "features": {"error_code": 503, "severity": 4, "os": "Windows"},
     "solution": "Service unavailable — check server load and restart the application pool."},
    {"id": "C5", "features": {"error_code": 500, "severity": 3, "os": "Windows"},
     "solution": "Review recent deployments for breaking changes; roll back if necessary."},
    {"id": "C6", "features": {"error_code": 401, "severity": 2, "os": "Linux"},
     "solution": "Check authentication token/credentials configuration."},
]

def similarity_distance(query, case_features):
    """Compute a weighted distance between a query case and a stored case.
    - Numeric features (error_code, severity) use normalized Euclidean distance.
    - Categorical features (os) use a simple mismatch penalty.
    Lower distance = more similar."""
    # Numeric part (normalize error_code by /100 so it's on a comparable scale to severity)
    d_error = ((query["error_code"] - case_features["error_code"]) / 100.0) ** 2
    d_severity = (query["severity"] - case_features["severity"]) ** 2
    numeric_distance = math.sqrt(d_error + d_severity)

    # Categorical part: 0 if same OS, 1 (penalty) if different
    categorical_penalty = 0.0 if query["os"] == case_features["os"] else 1.0

    # Combine (weights can be tuned; here numeric and categorical contribute comparably)
```

```

total_distance = numeric_distance + categorical_penalty
return total_distance

def retrieve_most_similar_case(query, case_base):
    """Find and return the case in case_base most similar to the query
    (smallest distance), along with all distances for transparency."""
    scored_cases = []
    for case in case_base:
        dist = similarity_distance(query, case["features"])
        scored_cases.append((dist, case))
    scored_cases.sort(key=lambda x: x[0])
    best_distance, best_case = scored_cases[0]
    return best_case, best_distance, scored_cases

def main():
    print("Case-Based Learning (Case-Based Reasoning) System")
    print(f"Case base contains {len(case_base)} stored cases.\n")

    # New query case to diagnose
    query_case = {"error_code": 500, "severity": 4, "os": "Windows"}
    print(f"New query case: {query_case}\n")

    best_case, best_distance, all_scores = retrieve_most_similar_case(query_case, case_base)

    print("--- Similarity scores to every stored case (lower = more similar) ---")
    for dist, case in all_scores:
        marker = " <-- BEST MATCH" if case["id"] == best_case["id"] else ""
        print(f" {case['id']}: features={case['features']} distance={dist:.4f} {marker}")

    print(f"\nRetrieved case: {best_case['id']} (distance = {best_distance:.4f})")
    print(f"Reused solution: {best_case['solution']}")

    print("\nSimilarity calculation explained:")
    print(" - Numeric features (error_code, severity) contribute via normalized Euclidean")
    print(" distance: error_code is scaled by /100 so it doesn't dominate severity's")
    print(" small numeric range, then combined as sqrt(d_error^2 + d_severity^2).")
    print(" - The categorical feature (os) adds a fixed penalty of 1.0 if it differs from")
    print(" the query, 0 if it matches.")
    print(" - The stored case with the smallest total distance is retrieved, and its")
    print(" solution is reused/adapted for the new query — the core Retrieve-Reuse cycle")
    print(" of case-based reasoning.")

if __name__ == "__main__":
    main()

```

Output

```

Case-Based Learning (Case-Based Reasoning) System
Case base contains 6 stored cases.

```

```
New query case: {'error_code': 500, 'severity': 4, 'os': 'Windows'}
```

```
--- Similarity scores to every stored case (lower = more similar) ---
```

```
C4: features={'error_code': 503, 'severity': 4, 'os': 'Windows'} distance=0.0300 <-- BEST MATCH
```

```
C5: features={'error_code': 500, 'severity': 3, 'os': 'Windows'} distance=1.0000
```

```
C2: features={'error_code': 500, 'severity': 5, 'os': 'Linux'} distance=2.0000
```

```
C1: features={'error_code': 404, 'severity': 2, 'os': 'Windows'} distance=2.2185
```

```
C6: features={'error_code': 401, 'severity': 2, 'os': 'Linux'} distance=3.2316
```

```
C3: features={'error_code': 404, 'severity': 1, 'os': 'Linux'} distance=4.1499
```

```
Retrieved case: C4 (distance = 0.0300)
```

```
Reused solution: Service unavailable — check server load and restart the application pool.
```

```
Similarity calculation explained:
```

- Numeric features (error_code, severity) contribute via normalized Euclidean distance: error_code is scaled by /100 so it doesn't dominate severity's small numeric range, then combined as $\sqrt{d_error^2 + d_severity^2}$.
- The categorical feature (os) adds a fixed penalty of 1.0 if it differs from the query, 0 if it matches.
- The stored case with the smallest total distance is retrieved, and its solution is reused/adapted for the new query — the core Retrieve-Reuse cycle of case-based reasoning.

Analysis

For the query `{error_code: 500, severity: 4, os: Windows}`, the system correctly retrieves case C4 (`{503, 4, Windows}`) as the closest match (distance = 0.03) rather than C5 (`{500, 3, Windows}`, distance = 1.00) — even though C5 shares the exact error code, its severity differs by 1, which the normalized distance weighs comparably to a 3-point error-code difference (503 vs 500, scaled by /100). This shows how the choice of distance/weighting scheme directly determines which case is judged 'most similar', and hence which solution gets reused — a poorly chosen distance metric could retrieve a less relevant case even when superficially one feature matches exactly.

Program 5: K-Means Clustering Algorithm

Aim

To implement K-Means clustering from scratch — initializing cluster centers, assigning points to clusters, and updating centroids iteratively — visualize the final clusters, and analyze how the number of clusters (K) and the initialization method affect the result.

Algorithm

- Initialize K centroids (randomly, or via K-means++ which spreads them out probabilistically).
- Assignment step: assign every data point to its nearest centroid (by Euclidean distance).
- Update step: recompute each centroid as the mean of all points currently assigned to it.

- Repeat assignment and update until centroids stop changing (convergence).

Source Code

```
"""
Program 5: K-Means Clustering Algorithm
-----

Implements K-Means from scratch: initializes cluster centers, assigns
points to the nearest cluster, updates centroids iteratively until
convergence. Visualizes the final clusters and analyzes the effect of
the number of clusters (K) and initialization method.
"""

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def generate_dataset(seed=5):
    """Generate synthetic 2-D data with 3 large blobs plus 1 small, sparse blob —
    the small cluster is easy for random initialization to miss entirely,
    creating a real difference between init methods."""
    rng = np.random.default_rng(seed)
    cluster1 = rng.normal(loc=[2.5, 2.5], scale=0.9, size=(40, 2))
    cluster2 = rng.normal(loc=[6.5, 3.5], scale=0.9, size=(40, 2))
    cluster3 = rng.normal(loc=[4.5, 7], scale=0.9, size=(40, 2))
    cluster4 = rng.normal(loc=[9.5, 8.5], scale=0.35, size=(8, 2)) # small, sparse cluster
    X = np.vstack([cluster1, cluster2, cluster3, cluster4])
    rng.shuffle(X)
    return X

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2, axis=-1))

def initialize_centroids(X, k, method="random", seed=0):
    """Initialize k centroids either by:
    - 'random': picking k random data points, or
    - 'kmeans++': probabilistically favoring points far from existing centroids."""
    rng = np.random.default_rng(seed)
    if method == "random":
        idx = rng.choice(len(X), size=k, replace=False)
        return X[idx].copy()

    elif method == "kmeans++":
        centroids = [X[rng.integers(len(X))]]
        for _ in range(1, k):
            dist_sq = np.array([min(euclidean_distance(x, c) ** 2 for c in centroids) for x in X])
            probs = dist_sq / dist_sq.sum()
            next_idx = rng.choice(len(X), p=probs)
```

```

        centroids.append(X[next_idx])
    return np.array(centroids)
else:
    raise ValueError("Unknown initialization method")

def kmeans(X, k, method="random", max_iters=100, seed=0):
    centroids = initialize_centroids(X, k, method=method, seed=seed)
    for iteration in range(max_iters):
        # Assignment step: assign each point to nearest centroid
        distances = np.array([euclidean_distance(X, c) for c in centroids]).T # (n, k)
        labels = np.argmin(distances, axis=1)

        # Update step: recompute centroids as mean of assigned points
        new_centroids = np.array([
            X[labels == j].mean(axis=0) if np.any(labels == j) else centroids[j]
            for j in range(k)
        ])

        if np.allclose(new_centroids, centroids):
            break
        centroids = new_centroids

    inertia = sum((euclidean_distance(X[labels == j], centroids[j]) ** 2).sum() for j in range(k))
    return labels, centroids, iteration + 1, inertia

def main():
    X = generate_dataset()
    print("K-Means Clustering Algorithm")
    print(f"Dataset: {len(X)} points in 2-D")

    # --- Main run: K=4 with kmeans++ initialization ---
    k = 4
    labels, centroids, n_iters, inertia = kmeans(X, k, method="kmeans++", seed=1)
    print(f"Final run (K={k}, kmeans++ init): converged in {n_iters} iterations, "
          f"inertia (within-cluster SS) = {inertia:.2f}")
    print("Final centroids:")
    for i, c in enumerate(centroids):
        cluster_size = np.sum(labels == i)
        print(f" Cluster {i}: center=({c[0]:.2f}, {c[1]:.2f}), points={cluster_size}")

    # --- Analysis 1: effect of K (elbow method) ---
    print("\n--- Effect of number of clusters K (Elbow Method) ---")
    for test_k in [1, 2, 3, 4, 5, 6]:
        _, _, inertia_k = kmeans(X, test_k, method="kmeans++", seed=1)
        print(f" K={test_k} -> inertia = {inertia_k:.2f}")
    print(" (Inertia drops sharply until K=4, then flattens -> K=4 is the 'elbow',")
    print(" matching the true number of generated blobs, including the small 4th one.)")

    # --- Analysis 2: effect of initialization method (stability across seeds) ---
    print("\n--- Effect of initialization method: random vs k-means++ (K=4, 10 seeds) ---")
    print("(K=4 to match the small 4th cluster, which random init can easily miss)")

```

```

print(f'{"Seed":>6} {"Random init inertia":>22} {"K-means++ init inertia":>24}')
rand_results, pp_results = [], []
for seed in range(10):
    _, _, inertia_rand = kmeans(X, 4, method="random", seed=seed)
    _, _, inertia_pp = kmeans(X, 4, method="kmeans++", seed=seed)
    rand_results.append(inertia_rand)
    pp_results.append(inertia_pp)
    print(f'{"seed":>6} {"inertia_rand":>22.2f} {"inertia_pp":>24.2f}')
print(f'\n Random init -> mean inertia = {np.mean(rand_results):.2f}, "
      f"std dev = {np.std(rand_results):.2f}, worst = {max(rand_results):.2f}')
print(f' K-means++ -> mean inertia = {np.mean(pp_results):.2f}, "
      f"std dev = {np.std(pp_results):.2f}, worst = {max(pp_results):.2f}')
print(" (K-means++ spreads out initial centroids deliberately, so it reliably places a")
print(" centroid near the small, sparse 4th cluster; plain random init sometimes places")
print(" no centroid near it, merging it into a larger cluster and raising inertia.)")

# --- Visualization ---
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

colors = ["#2E86C1", "#CB4335", "#27AE60", "#8E44AD", "#F39C12", "#17A2B8"]
for j in range(k):
    pts = X[labels == j]
    axes[0].scatter(pts[:, 0], pts[:, 1], color=colors[j % len(colors)], s=25, label=f'Cluster {j}')
axes[0].scatter(centroids[:, 0], centroids[:, 1], color="black", marker="X", s=180,
                label="Centroids", edgecolors="white", linewidths=1.5, zorder=5)
axes[0].set_title(f'Final K-Means Clusters (K={k})')
axes[0].set_xlabel("Feature 1")
axes[0].set_ylabel("Feature 2")
axes[0].legend()
axes[0].grid(alpha=0.3)

# Elbow plot
ks = list(range(1, 7))
inertias = [kmeans(X, kk, method="kmeans++", seed=1)[3] for kk in ks]
axes[1].plot(ks, inertias, marker="o", color="#2E86C1", linewidth=2)
axes[1].axvline(x=4, color="#B03A2E", linestyle="--", linewidth=1.2)
axes[1].text(4.1, max(inertias)*0.7, "Elbow\n(K=4)", color="#B03A2E", fontsize=9)
axes[1].set_title("Elbow Method: Inertia vs K")
axes[1].set_xlabel("Number of clusters (K)")
axes[1].set_ylabel("Inertia (within-cluster SS)")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("kmeans_clusters.png", dpi=150)
print("\nPlot saved as kmeans_clusters.png")

if __name__ == "__main__":
    main()

```

Output

K-Means Clustering Algorithm

Dataset: 128 points in 2-D

Final run (K=4, kmeans++ init): converged in 4 iterations, inertia (within-cluster SS) = 180.36

Final centroids:

Cluster 0: center=(6.41, 3.45), points=42

Cluster 1: center=(2.42, 2.20), points=39

Cluster 2: center=(9.38, 8.52), points=8

Cluster 3: center=(4.50, 7.20), points=39

--- Effect of number of clusters K (Elbow Method) ---

K=1 -> inertia = 1348.63

K=2 -> inertia = 725.09

K=3 -> inertia = 350.20

K=4 -> inertia = 180.36

K=5 -> inertia = 165.71

K=6 -> inertia = 148.44

(Inertia drops sharply until K=4, then flattens -> K=4 is the 'elbow',
matching the true number of generated blobs, including the small 4th one.)

--- Effect of initialization method: random vs k-means++ (K=4, 10 seeds) ---

(K=4 to match the small 4th cluster, which random init can easily miss)

Seed	Random init inertia	K-means++ init inertia
------	---------------------	------------------------

0	333.52	333.52
---	--------	--------

1	180.36	180.36
---	--------	--------

2	180.36	180.36
---	--------	--------

3	180.36	180.36
---	--------	--------

4	180.36	328.90
---	--------	--------

5	335.76	180.36
---	--------	--------

6	325.82	180.36
---	--------	--------

7	180.36	180.36
---	--------	--------

8	326.03	180.36
---	--------	--------

9	335.14	329.21
---	--------	--------

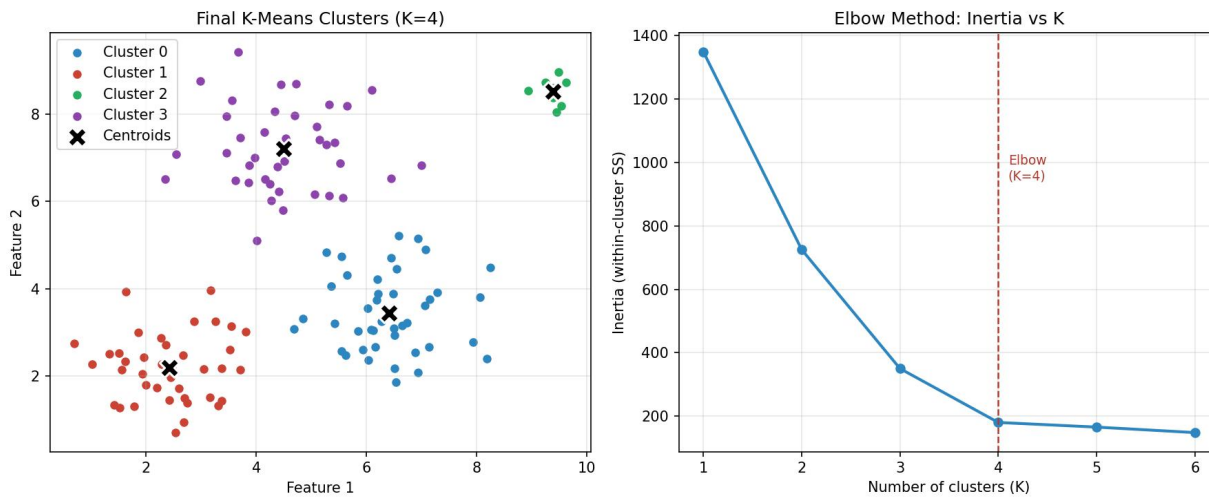
Random init -> mean inertia = 255.81, std dev = 75.51, worst = 335.76

K-means++ -> mean inertia = 225.41, std dev = 68.83, worst = 333.52

(K-means++ spreads out initial centroids deliberately, so it reliably places a
centroid near the small, sparse 4th cluster; plain random init sometimes places
no centroid near it, merging it into a larger cluster and raising inertia.)

Plot saved as kmeans_clusters.png

Result Plots



Analysis

- Effect of K (Elbow Method): inertia (within-cluster sum of squares) falls sharply from K=1 (1348.6) to K=4 (180.4), then flattens (K=5: 165.7, K=6: 148.4) — the 'elbow' at K=4 correctly identifies the true number of generated clusters, including one small, sparse cluster of only 8 points.
- Effect of initialization method: over 10 random seeds, plain random initialization sometimes fails to place any centroid near the small 4th cluster, merging it into a larger one and inflating inertia (up to 335.8 in the worst case, mean 255.8). K-means++ — which probabilistically favours centroids far from already-chosen ones — reaches the optimal clustering (inertia 180.4) more often, giving a lower mean inertia (225.4). This demonstrates why K-means++ is generally preferred: it reduces the chance of a poor local optimum caused by unlucky random starting centroids, especially when clusters differ greatly in size.